# Three Ways to Learn Repair Energies

A detailed, plain-language explanation of LOCAL-SEQUENCE, JOINT-CAUSAL-EBM, and HYBRID-JOINT

Latent Causal Infeasibility for Long-Horizon TAMP | 2D learning study

21 October 2026

## Contents


**1 Start here: the idea in two minutes** **2**
1.1 What is already real, and what is not? . . . 3

**2 What exactly is the robot's problem?** **3**
2.1 The physical environment is not a toy point robot . . . 3
2.2 What is a repair? . . . 3
2.3 Three separate questions . . . 4

**3 What information does the learner see?** **4**
3.1 Allowed model features, known before acting . . . 4
3.2 Forbidden model features . . . 4
3.3 Which parts of the system are actually learned? . . . 5

**4 Where do labels come from, and why is this not simply RL?** **5**
4.1 Four statuses after the P4 policy amendment . . . 5
4.2 What is measured for each repair? . . . 5
4.3 First-witness robot-motion cost . . . 6

**5 What is an energy model, in ordinary terms?** **6**
5.1 Where does Boltzmann enter? . . . 7
5.2 The two sorts of energy we must distinguish . . . 7

**6 Shared building blocks: object encoding and local energies** **7**

**7 Architecture A — LOCAL-SEQUENCE: 'learn a repair score from local compatibility and order'** **8**
7.1 The intuition . . . 8
7.2 Exact information flow . . . 8
7.3 The mathematical energy . . . 8
7.4 How A predicts an explanation . . . 8
7.5 Example . . . 9
7.6 Pros, limitations and decisive test . . . 9

**8 Architecture B — JOINT-CAUSAL-EBM: 'learn one score for a repair and its hypothetical outcomes together'** **9**
8.1 Why try a second architecture? . . . 9
8.2 What are the binary $z$ variables? . . . 9
8.3 The joint energy and what its factors mean . . . 10
8.4 What does 'jointly learned' mean? . . . 10
8.5 A toy numerical example of why coupling could help . . . 11
8.6 How B handles missing or duplicated outcome fields . . . 11
8.7 How B predicts a repair's success . . . 11
8.8 How Boltzmann finds repairs under B . . . 12

**9 Architecture C — HYBRID-JOINT: 'B's explanation model plus A's ordering memory' 12**
9.1 Why C might win . . . 12
9.2 Why C might lose . . . 13

**10 What is optimized? Every loss in plain language 13**
10.1 Local static loss: 'did I estimate geometry compatibility correctly?' . . . 13
10.2 Full ordered task loss: 'does this complete repair work?' . . . 13
10.3 Effect loss: 'did I understand what the intervention changes?' . . . 13
10.4 Travel-cost loss: 'how far did the actual robot joints move?' . . . 13
10.5 A's sequence-only pool energy loss . . . 14
10.6 B/C's true joint Boltzmann loss . . . 14
10.7 Confidence, class balance and gradients . . . 14

**11 One worked example from initial scene to gradient 14**

**12 Displacement, ordering and robot travel: who decides what? 15**
12.1 Is moving objects learned? . . . 15
12.2 Is displacement cost learned? . . . 15
12.3 Exactly how is ordering used? . . . 16

**13 The sampler, the optimizer, and final plan selection are different things 16**
13.1 Sampler moves and mathematical validation . . . 16

**14 What happens when the robot actually acts? The one-shot boundary 17**

**15 The actual A-versus-B-versus-C experiments 17**
15.1 Primary metrics . . . 18

**16 What new data do we need before the first neural proof of concept? 18**

**17 Exactly what would the proof-of-concept phase do? 19**

**18 Common misconceptions and how to answer them 20**

**19 Decision and next implementation order 21**

**20 Appendix A. Exact source basis and status 21**

**Document status.** Explanatory research report and proposed next-step plan, **not** a claim of model training or a new experiment. The scientific architecture is grounded in *Methodology v1.7* and *Implementation Guide v1.9* (19 October 2026). The **later** policy amendment `2D-ABC-OPNEG-v1`, implemented and measured in report **0019** (21 October 2026), authorizes rigorously recorded, fixed-budget **operational** negatives for **new** data. That policy overrides v1.7's original proof-only negative requirement *for those newly versioned teacher records*, but it does not turn planner failure into mathematical impossibility. The B/C outcome-layout correction described here is **proposed, not yet implemented**. No source code or new teacher data were available to inspect beyond the supplied reports.

# 1 Start here: the idea in two minutes

A robot is given an **initial scene**, a **target object**, a **goal destination**, and a set of **objects it is allowed to relocate**. Its job is to choose a *short*, *ordered* list of object moves so that it can then accomplish the original target action.

Think about a workbench. A cup needs to reach its goal. Moving a nearby block might make it possible. Sometimes one block must be moved *before* another. Some moves help, others do nothing, and some make the task worse. The robot must choose **what to move, where to move it, and in what order**. It must also avoid unnecessarily long robot-arm motion.

We are testing three answers to one modeling question: **What form of learned energy best captures which complete corrective plans will work and why?**

- **A — LOCAL-SEQUENCE:** Learn a set of local compatibility penalties and one order-sensitive GRU score for the complete proposed sequence. Get explanations by **re-scoring** the model after hypothetical edits or omitted actions. This is the simplest of the three proposed architectures.
- **B — JOINT-CAUSAL-EBM:** Learn **one joint energy** for a proposed repair **and** a small, explicit list of hypothesized outcomes under interventions and omissions. Its learned factors connect those outcomes to the repair. It models action order with learned prefix/pair interactions, **not** a GRU.
- **C — HYBRID-JOINT:** Keep B's explicit hypothesized outcomes and joint couplings, but use A's order-sensitive GRU for the repair sequence. This asks whether the extra causal representation helps even after giving the model a strong order representation.

**Energy** is simply a number the model assigns: generally **lower means more preferred/compatible**. It is **not** necessarily physical electrical energy, and it is **not** an automatic guarantee that a robot path exists.

**Teacher** means the external robot solver/checker that generates labels and verified executions during offline dataset creation. **Learner** means the neural network; it sees the initial scene and proposed hypothetical repairs but not the teacher's answer.

### 1.1 What is already real, and what is not?

As of report 0019, **P4 is complete**: 261/261 regression tests and six archived replays passed; 12/12 development queries were decided; 5 were witnessed feasible, 4 were certified infeasible, and 3 were fixed-budget operational negatives. The entire P4 pilot used 77.6 CPU-seconds and involved **two selected development roots**. There is **one fully observed operational B/C outcome pool**, recorded as eight slots but containing **seven distinct outcomes** because one omission slot duplicates the full repair. An important access-based order contrast from `g4c_r19` is independently grounded in both a proof and a witness.

**Not done yet:** the corrected B/C layout, real A/B/C neural training, a meaningful multi-root quantitative comparison, and the large-scale 2D or 3D evaluation. A tiny smoke test is currently feasible; a research conclusion about which model wins is not.

## 2 What exactly is the robot's problem?

### 2.1 The physical environment is not a toy point robot

The current experiment uses a fixed-base **three-joint planar arm**, including the finite-width rigid links, housings, base, palm, two fingers, and an object rigidly held while carrying. The code uses continuous joint paths and continuous collision checks. The scene contains disc objects and an **8 by 8 grid of supported resting positions** at 0.08-m pitch. The grid makes **rest destinations** discrete; it does not mean the arm teleports or travels on grid edges. The existing declared manipulation skill has **48 grasp candidates**, representing 24 tool-axis orientations and two elbow branches. A target is successful only with the goal achieved and a checked **SAFE_RETREAT**.

The input is intentionally favorable and controlled: initial RGB-D and instance masks, supplied object poses and simple shapes, robot angles/limits and gripper state, task/goal information, and legal candidate destinations. This does **not** assume that the network learns the robot kinematics from raw pixels. We supply geometry because the scientific question is about **learning repair decisions**, not about inventing perception and low-level control simultaneously.

### 2.2 What is a repair?

A repair is a list such as

$$r = [(\text{A}, \text{slot7}), (\text{B}, \text{slot21})].$$

This means: move object A completely to slot 7; from the *actual achieved robot state*, move B completely to slot 21; then try the original target task. The sequence order matters. Each object may appear only once and each destination must be a legal physical placement. The empty list [ ] means **do not relocate anything**.

The teacher must try the **fixed specified order**. It can choose among permitted grasps and continuous joint paths, but it must not silently switch A then B into B then A. After each relocation the actual arm configuration and gripper state carry forward. No magical reset to the start pose is allowed.

**Crucial:** One successful trajectory for A then B does **not** mean B then A is feasible. A successful hypothetical object teleport also does **not** prove that the robot can execute that teleport as a relocation.

### 2.3 Three separate questions

**Feasibility:** Can the robot complete the target task after this exact ordered repair under its allowed execution domain or agreed operational teacher protocol?

**Explanatory effect:** If we edit one object's initial pose *without moving the robot*, does the original task outcome change? If we remove one action from a particular repair and replay from the initial scene, does the outcome change?

**Cost:** How much total robot joint travel did the first complete relocation witness require? This is **not** the same as how far the object moved on the table, and is not a claim of physical electrical energy.

All three architectures face the **same** questions, candidate repairs, root splits, evidence semantics, and final robot verifier.

## 3 What information does the learner see?

There are two information streams; mixing them up would invalidate the experiment.

### 3.1 Allowed model features, known before acting

1. **Initial appearance.** A frozen DINOv2 feature extractor (or a documented frozen equivalent) takes initial image features. Instance-mask pooling creates a vector for each visible object. A small trainable projection then transforms each vector into the shared **128-dimensional object embedding**.
2. **Known geometry.** Actual supplied object pose, primitive shape, size, support and candidate position. These are numeric model inputs; we are not requiring the image encoder to infer known ground truth again.
3. **Known robot state.** Initial joint angles, joint limits, forward-kinematic context, gripper geometry, gap, attachment status and goal specification.
4. **The hypothetical decision.** The object's identity, its proposed destination, and its position in the proposed action order.
5. **Ideal hypothetical scene prefixes.** To score the second move, the model can represent the first object's *requested resting destination* and all untouched objects at their initial resting places. These are bookkeeping hypotheses, **not** privilege-bearing future observations or predicted robot trajectories.
6. **Fixed, predeclared explanation query catalogue.** Which valid scene edits and omissions are eligible to compare can be supplied. The *outcomes* of those queries cannot.

### 3.2 Forbidden model features

The learner is **not** given the teacher's complete robot trajectory, target swept volume, collision checker output, RRT seed, grasp chosen by the successful teacher, future robot state, intervention outcome, outcome label, or hidden

identifiers revealing a training family/root. These are used to **make or audit labels**, not to predict them.

This is especially important because our geometry is supplied. Feeding a precomputed collision answer as a feature would make an apparently impressive network that merely copies the teacher.

### 3.3 Which parts of the system are actually learned?

**Frozen / analytic:** DINOv2 backbone initially, FK/IK, collision geometry, legality of a resting destination, exact displacement formula, move count, robot teacher, geometric certificates, search kernels, and teacher policy. Temperature and loss weights are selected using validation rather than learned as hidden physics.

**Trainable:** Object-feature projection, local unary and symmetric pair MLPs, sequence-reasoning factors (GRU in A/C; prefix interaction potentials in B), B/C intervention and omission potentials, B/C coupling potentials, feasibility predictions and measured joint-travel predictors. All three models use the same target labels and compatible feature schema.

## 4 Where do labels come from, and why is this not simply RL?

The model is learned **offline from independently evaluated candidate actions**. It is not directly trained by reinforcement learning, not differentiating through the robot motion planner, and not learning a joint-velocity controller.

For each planned query, the teacher starts at the same immutable initial scene, attempts the specified repair, records its relocation trajectory if successful, and then evaluates the original target action from the achieved state. Scene edits are evaluated by editing the initial object poses while leaving the robot fixed; omissions are **fresh repaired queries** with one or more steps removed.

### 4.1 Four statuses after the P4 policy amendment

- **`VERIFIED_FEASIBLE` = positive.** A complete independent witness exists, including the required safe retreat.
- **`CERTIFIED_INFEASIBLE` = negative, proof-grounded.** An independent checker has validated a mathematical/geometric certificate for the exact scope.
- **`BUDGET_INFEASIBLE` = negative, operational.** All prescribed planner seeds completed with the named clean terminal outcomes and no witness. Under report 0019's **`2D-ABC-OPNEG-v1`** policy the three seed namespaces are `20261021`, `20261022`, `20261023`; guards and clean terminals are pinned. This means *the declared protocol did not solve the query*, **not** that no physically possible execution exists.
- **`UNKNOWN` = not a binary label.** Aborted, censored, corrupted, incomplete, incompatible, or otherwise undecidable under the actual policy. Preserve the audit record; mask it out of supervised losses.

These are **evidence types**, not interchangeable scientific claims. Do not retroactively rewrite pre-P4 historical UNKNOWNs. Use a **versioned projection** from raw records into compatible training examples. Do not combine effects from incompatible policy hashes.

**Critical limitation of P4:** its three budget negatives were seed-invariant deterministic screen/sweep failures, not informative examples of diverse RRT searches. A fixed large computational budget is a *measurement convention*, not an asymptotic theorem.

### 4.2 What is measured for each repair?

Let $Y(r)$ be the teacher's binary outcome when decided. The empty repair gives the **original-task outcome** $Y_0 = Y([])$. For the same scene, two ordered repairs with exactly the same final slots can have different labels. Local static candidate-pose/pose-pair labels are also available when decided.

For a scene edit $I$, temporarily assign an eligible object to a different supported resting pose **while holding the robot's initial state fixed**. Evaluate the original target **without any robot relocation**. If both outcomes were decided:

$$\Delta_I^S = Y_I - Y_0.$$

Example: original task fails (0), edited scene succeeds (1), so the effect is **+1**. This measures the outcome difference for that **specific predeclared edit**. It does not prove that the edit can be performed by the robot.

For an omission from a repair $r$, physically rebuild and retry the shorter sequence $r \setminus A$ from the original scene. If both are decided:

$$\Delta_A^O = Y(r) - Y(r \setminus A).$$

Example: A then B works (1), removing A makes B-alone fail (0); omission effect is **+1**. This proves A mattered **within this particular repair and protocol**, not necessarily across every imaginable repair.

For two compatible scene edits $I, J$, if all four original-task outcomes exist:

$$\kappa_{I,J} = Y_{I \cup J} - Y_I - Y_J + Y_0.$$

- $\kappa = 0$: no measured nonlinear interaction for those binary outcomes.
- $\kappa = +1$ could indicate that both edits together succeed although neither individually is enough (depending on the four values).
- $\kappa = -1$ can indicate redundancy or other interaction in binary outcomes.

These are **observed effect contrasts**, not necessarily hidden real-world causal edges. If a side uses a `BUDGET_INFEASIBLE` label, mark the effect **operational** rather than proof-grounded.

### 4.3 First-witness robot-motion cost

If there is a *complete relocation witness*, we measure normalized total joint motion across all relocation phases and inter-action transitions:

$$J(w) = \sum_{s,j} \frac{|q_{s+1,j} - q_{s,j}|}{q_j^{\max} - q_j^{\min}}.$$

The first complete relocation witness fixes this cost label **before inspecting whether the target succeeds**. Thus a proposal can have a **cost label but a negative final task label**. If no complete relocation witness exists, it has no cost label. The empty repair's relocation cost is exactly zero. Scene teleports/edits have **no relocation cost supervision**. We do not infer cost from object displacement or from the cheapest of many target-selected trajectories.

## 5 What is an energy model, in ordinary terms?

Imagine a teacher grading several proposed repair plans. Instead of selecting one plan directly, a neural model learns to assign each plan a **compatibility bill**. A lower bill means the plan fits the observed scene and known task constraints more naturally.

A bill may contain several terms: this destination seems unsuitable, these two final resting positions conflict, the action sequence is likely to fail, and a particular hypothetical causal outcome is inconsistent with the rest. The model learns numerical parameters in those terms from examples.

A learned **energy** is not a theorem. A plan with energy 0.1 can still fail robot verification. The external checker is always the final source of actual execution validity.

### 5.1 Where does Boltzmann enter?

Energy gives a preference distribution. At temperature $T > 0$,

$$P(\text{candidate}) \propto \exp[-E(\text{candidate})/T].$$

A candidate with low energy is drawn more often; a higher-temperature sampler explores more alternatives. **Boltzmann search is a search/inference tool**. We are not claiming to train a separate classical Restricted Boltzmann Machine or a Hopfield memory network.

Here is a purely illustrative probability calculation at $T = 1$: if three plan energies are 0, 1 and 2, their unnormalized weights are 1, 0.368 and 0.135. The normalized probabilities are approximately **0.665, 0.245 and 0.090**. The model could still be wrong; those are just its own learned preferences.

### 5.2 The two sorts of energy we must distinguish

**Feasibility energy:** What the model learns to associate with an executable, task-completing repair. It is used in the pool-level energy training losses.

**Sampling energy:** Feasibility energy **plus** practical preferences about displacement and predicted robot travel. Used to suggest plans efficiently.

Do **not** conflate them. A cheap movement is not necessarily a valid repair, and a valid repair need not be cheap.

## 6 Shared building blocks: object encoding and local energies

All three architectures use the same fundamental scene and local-compatibility representation.

**Object vector:** For object $i$, concatenate/merge frozen mask-pooled visual appearance, its supplied geometry and pose, candidate destination data, support relationships, and task/robot context; transform through shared learned layers into a width-128 object token.

**Unary potential $U_i$:** A learned estimate of whether one object's requested endpoint is sensible. For example, does the proposed cell fit the object and its support, according to the patterns seen during training?

**Pair potential $V_{ij}$:** A learned estimate of whether two *simultaneous endpoints* are compatible. For example, do two objects compete for the same resting space? Pair factors must be symmetric in their objects; reordering the object array must not change their physical interpretation.

The small neural heads emit logits $\ell$. For probability $q = \sigma(\ell)$, a simple nonnegative energy conversion is

$$e = -\log q = \text{softplus}(-\ell).$$

The robot's exact geometry checker still rejects obviously invalid endpoints independently. The learned local factors can help rank proposals without invoking expensive full robot motion search for each one. They are **not replacements for the checker**.

# 7 Architecture A — LOCAL-SEQUENCE: 'learn a repair score from local compatibility and order'

## 7.1 The intuition

Imagine A as an experienced warehouse supervisor. It reads the scene, checks each proposed final resting position and each pair of final positions, and then reads the complete action list **in order**, remembering what has already been proposed.

A is asked: 'If we move B out of the way first, then move A, will the target task work?' It learns to distinguish that from 'Move A first, then B.' It also predicts how much joint travel the relocations will take.

But when asked *why* an intervention mattered, it does **not** have special cause/outcome variables. It re-scores the original scene and the hypothetically edited scene and compares its own predicted probabilities.

## 7.2 Exact information flow

1. Encode the **initial** objects, robot and target.
2. Build the proposed final endpoint assignment $x(r)$.
3. Evaluate shared learned unary $U_i$ and pair $V_{ij}$ penalties.
4. Build ideal **prefix** scenes: after zero requested relocations, after the first, after the second, etc. Every object remains present.
5. For each action, feed its object/destination embedding and the corresponding all-object prefix summary to a width-128 **GRU**. A GRU is a small recurrent memory: it updates a vector after every proposed step so earlier proposed actions can affect the meaning of later ones.
6. The final GRU output generates task-feasibility logit $u_A(r)$ and nonnegative travel prediction $\widehat{J}_A(r)$.

## 7.3 The mathematical energy

$$E_A^{\text{feas}}(r) = \underbrace{\sum_i U_A(x_i(r))}_{\text{individual endpoints}} + \underbrace{\sum_{i<j} V_A(x_i(r), x_j(r))}_{\text{endpoint pairs}} + \underbrace{\lambda_t \,\text{softplus}[-u_A(r)]}_{\text{ordered whole-task factor}}.$$

The associated task probability-like output is

$$p_A(r) = \sigma(u_A(r)).$$

**Important:** the summed feasibility energy and the GRU task probability are related but are **not the same numeric quantity**. During the PoC, verify both the task-logit BCE calibration and finite-pool energy ranking; do not assume a perfect monotonic correspondence in every context simply from sharing parameters.

**What A is learning:** local endpoint compatibility, interactions between endpoints, a higher-order ordered plan-to-success mapping, and a relocation joint-travel approximation.

**What A is *not* learning:** explicit distributions over intervention outcomes, a causal DAG, the exact robot IK or collision rules, and the physics of every possible grasp.

## 7.4 How A predicts an explanation

For a fixed-robot scene edit $I$, score the original empty-repair query and the changed-scene empty-repair query:

$$\widehat{\Delta}^S_{A,I} = p_A([] \mid S_0^I) - p_A([] \mid S_0).$$

For omission of action $t$ from repair $r$, independently re-encode/rescore the shorter ideal sequence:

$$\widehat{\Delta}^{O}_{A,t} = p_A(r \mid S_0) - p_A(r \setminus t \mid S_0).$$

This is **implicit explanation**: A's same scoring network is reused for the contrasted scenarios. There is no separate trained table of 'A caused the failure'.

### 7.5 Example

For the real `g4c_r19` pilot, moving `o00` first is **certified inaccessible**, while first moving `o01` and then `o00` has an execution witness. A's GRU can in principle learn that the two orderings differ despite having identical endpoints. The tiny-data objective is to make its task probability and energy ranking reflect that difference **on development examples**.

### 7.6 Pros, limitations and decisive test

**Advantage:** relatively direct, easier to train with individually decided sequence labels; it can exploit ordering without having complete intervention sets on every candidate.

**Limitation:** explanation consistency is not architecturally forced. Re-scoring two hypothetical scenes independently can lead to effects that do not agree neatly with a coherent joint distribution over all proposed outcomes.

**Test:** Can A memorize real decided positive/negative repair pairs, distinguish same-endpoint order swaps, predict measured J, and rank successful repairs before unsuccessful ones under equal search budgets? Compare against analytic geometry and an order-insensitive learned ablation. **Success would establish functioning order-sensitive learning, not new causal identifiability.**

## 8 Architecture B — JOINT-CAUSAL-EBM: 'learn one score for a repair and its hypothetical outcomes together'

### 8.1 Why try a second architecture?

A scores each question separately. Suppose we ask:

- Did the original target fail?
- If object A were moved in the initial scene *without moving the robot*, would the target succeed?
- If B were moved instead, what then?
- If both were edited, what then?
- Would the complete robot-executable ordered repair succeed?
- Would omitting its first action make it fail?

Those answers are linked. B's central hypothesis is that **learning their joint consistency** leads to better repair ranking and better effect predictions than computing each answer independently.

Analogy: A grades each essay on its own; B grades a complete **case file** that includes the proposed plan **and** the predicted outcomes of the related what-if cases. B rewards combinations of answers and plans that fit the data it has seen.

### 8.2 What are the binary $z$ variables?

For candidate repair $r$, define

$$z = (z_0, \{z_I^S\}, z_r, \{z_A^O\}).$$

These are **hypothetical outcomes**, not oracle labels fed into the model:

- $z_0$: Is the original task successful with no repair?
- $z_I^S$: Would the original task succeed under a particular **predeclared fixed-robot scene edit**?
- $z_r$: Would the original task succeed after executing the specific ordered repair?
- $z_A^O$: Would the task succeed after removing a specified step or subset of steps from that repair and reconstructing from the initial state?

Each is 0 or 1. During **training**, independently obtained teacher labels tell us which answers were observed. During **inference**, the model considers possible assignments and their probabilities. It must never receive the true teacher answers as input.

**Example with two relocations:** one original outcome, three scene-edit outcomes (A edit, B edit, joint edit), one full-repair outcome and two genuine leave-one-out outcomes = **seven distinct outcome variables**. This is the corrected intended P4/D3 versioned layout; the old stored eighth slot mistakenly repeated the full repair.

## 8.3 The joint energy and what its factors mean

$$\begin{aligned} E_B(r, z \mid c_0, Q) = {} & E_B^{\text{static}}(x(r), c_0) + \phi_B^{\text{seq}}(r, z_r, c_0) \\ & + \sum_I \phi_B^{\text{scene}}(I, z_0, z_I^S, c_0) \\ & + \sum_A \phi_B^{\text{omit}}(r, A, z_r, z_A^O, c_0) \\ & + \phi_B^{\text{couple}}(r, z, c_0). \end{aligned}$$

Read this like an itemized bill:

- **Static factor:** Do requested final object positions make sense?
- **Sequence factor:** Is this *ordered list* compatible with the proposition that its final target task succeeds or fails? B models order with learned **prefix-conditioned pair/adjacency** potentials. It does not use a GRU.
- **Scene factor:** Is the pair of hypothesized outcomes (before and after a specified fixed-robot edit) plausible given the scene and edit?
- **Omission factor:** Is the hypothesized outcome of the full repair compatible with the hypothesized result of omitting a particular action?
- **Coupling factor:** How compatible is the **whole repair choice** with the full bundle of hypothesized original, edited and omitted outcomes? This is the key extra trainable dependency that A lacks.

Every $\phi$ is a learned neural potential, not an if-statement that the programmer writes for a particular object. Large weights or factor-graph connections are **not causal arrows** and do not prove necessity.

## 8.4 What does 'jointly learned' mean?

B does **not** first train a classifier and then bolt a graph diagram or sampler onto it. It assigns one energy to every allowed **pair**:

$$(\text{candidate ordered repair},\ \text{hypothesized outcome configuration}).$$

It then defines the conditional Boltzmann model

$$P_B(r, z \mid c_0, Q) = \frac{\exp[-E_B(r, z \mid c_0, Q)/T]}{\sum_{v \in \mathcal{C}} \sum_{z' \in \mathcal{Z}(v)} \exp[-E_B(v, z' \mid c_0, Q)/T]}$$

**within the explicitly declared finite candidate catalogue** $\mathcal{C}$. Its potentials can be optimized *together* through a negative log-probability that looks at **repair choices and all possible hypothetical outcome assignments**. This is a true trained **joint energy-based formulation**, not an independent BCE classifier plus a fancy sampling name.

**Subtle but essential:** wrong hypothetical $z$ vectors in the denominator are **model hypotheses**, not fake teacher labels. The teacher must have decided every true active field for the candidate before we use that candidate in the exact joint-pool term.

## 8.5 A toy numerical example of why coupling could help

This example is invented solely to explain the method, **not a measured run**.

Suppose we have two actions and two possible orders:

- $r_1 = [B \to b, A \to a]$: teacher says it succeeds.
- $r_2 = [A \to a, B \to b]$: teacher says it fails.

Both have the same endpoint configuration. Local endpoint factors cannot distinguish them by themselves.

Now suppose the teacher's omission tests say removing B from $r_1$ makes it fail, while removing A from $r_1$ also makes it fail. B may learn that the successful repair outcome $z_{r_1} = 1$ is compatible with two omitted-repair outcomes $z^O = 0, 0$, and that this pattern is associated with a specific order and scene context. Its coupling energy should learn to prefer coherent bundles when properly supervised.

This **may** help on new scenes with analogous access patterns. But it may also add complexity without benefit. That is the scientific question.

## 8.6 How B handles missing or duplicated outcome fields

This is currently our most important architectural correctness issue.

Different repair lengths generate different eligible omission queries. B must use a **fixed maximum typed layout with an active mask**. Inactive slots are *not random binary dimensions*. If an inapplicable slot were allowed to take 0 or 1, it would double the number of hypothetical assignments for no reason and bias probability toward repairs with more inactive slots.

Moreover, an omission that **removes no action** is just the original repair; it must not become a second independent latent outcome variable. Report 0019 found exactly this bug in the frozen G4C layout: a two-move repair has a third 'omission' slot which does nothing. We must create **a versioned corrected view**, keep the old evidence immutable, and prove that the energy and partition function do not double-count it.

Further recommended audit: if two *different named queries* are physically identical (e.g., omitting the sole action equals the empty repair), preserve their provenance but specify how the model ties or constrains their outcome variables. Do not permit inconsistent joint hypotheses to gain probability solely from duplicate semantic identities. This is a **recommended model audit**, not a claim that report 0019 already fixed these aliases.

## 8.7 How B predicts a repair's success

Marginalize its hypothesized other outcomes:

$$p_B(r) = P_B(z_r = 1 \mid r, c_0, Q) = \sum_{z : z_r = 1} P_B(z \mid r, c_0, Q).$$

Similarly, intervention-effect estimates come from differences in **outcome marginals**, such as

$$\widehat{\Delta}^{S}_{B,I} = P_B(z^S_I = 1) - P_B(z_0 = 1).$$

No separate free-floating task classifier is permitted to disagree with the joint energy. The task BCE is applied to the **derived marginal logit**.

### 8.8 How Boltzmann finds repairs under B

The probability of a repair must sum over all valid hypothetical outcome configurations:

$$P_B(r) = \sum_z P_B(r, z), \quad \bar{E}_B(r) = -T \log \sum_z \exp[-E_B(r, z)/T].$$

At most eight active binary fields means at most 256 assignments, so **exact `logsumexp`** is practical in small PoC catalogues. We can then use a validated sampler over repairs scored by $\bar{E}_B(r)$.

It would be **wrong** to sample an outcome vector for repair 1, keep it fixed, and compare it directly with repair 2's joint energy when their active fields/meaning differ. The exact marginal (or a separately proven joint transition kernel) is required.

**Advantage:** explicit, probabilistically coherent structure for the predefined intervention/outcome variables, learnable coupling between explanations and repair choices.

**Disadvantages:** more data-hungry and computationally expensive; exact joint losses require complete decided outcome pools; architecture may overfit trivial intervention patterns; factor links do not magically discover causes.

## 9 Architecture C — HYBRID-JOINT: 'B's explanation model plus A's ordering memory'

C tries to combine what A and B do best.

Think of B as someone holding the **whole case file** of related 'what if' outcomes, but reading the order of actions mostly through local adjacent/prefix interactions. C holds the same case file but also has a **GRU memory** of the complete proposed action sequence.

Mathematically, C retains the static, scene, omission and coupling potentials of B, while replacing B's basic order factor with a GRU-based term:

$$E_C(r, z) = E^{\text{static}}_B + E^{\text{scene}}_B + E^{\text{omit}}_B + E^{\text{couple}}_B + \lambda_g \phi^{\text{GRU}}_C(r, z_r).$$

**Do not count order twice.** The comparison version should **replace**, not silently add on top of, B's previous order factor. An additive-B-order-plus-GRU version would be a separate named ablation.

C has a GRU-based relocation cost head, the same finite-pool joint likelihood and outcome marginalization as B, and the same scene/omission effect losses. Its inference distribution is genuinely over $(r, z)$, just as in B.

### 9.1 Why C might win

Consider a three-step dependency: move blocker A to a side shelf, then relocate B through newly opened access, then relocate C to free the target. The effect of moving C may depend on an interaction between **both earlier actions**. The GRU can maintain a richer learned history than B's deliberately constrained pair/prefix factors.

If the correct what-if outcome assignments also matter, the explicit joint z coupling may help C outperform A's independent re-scoring.

### 9.2 Why C might lose

Its additional representation can overfit a small dataset, increase gradient and inference complexity, and learn a spurious pattern related to the hand-built scene family. Its performance could improve merely because it has **more parameters**, not because it models explanations better.

This is why the primary comparison is **C versus A with the GRU held present**, plus **parameter-matched A** and **C without coupling**. Compare **B versus C** to examine the prefix-pair versus recurrent-order factor, but control model size and search compute.

## 10 What is optimized? Every loss in plain language

We minimize a **weighted sum of supervised errors** and, when the required complete candidate sets exist, an **energy-model probability loss**. Backpropagation updates the model's trainable weights using AdamW. The robot path planner is **not** differentiated through.

### 10.1 Local static loss: 'did I estimate geometry compatibility correctly?'

$$\mathcal{L}_{\text{static}} = \text{mean}_{\text{decided local factors}}\ \text{BCEWithLogits}(\ell_f, y_f).$$

BCE means binary cross-entropy: predicting 0.95 for a definitely invalid placement is bad; predicting 0.95 for a valid placement is good. Train unary and pair heads only on decided static labels. Local validity is not task feasibility.

### 10.2 Full ordered task loss: 'does this complete repair work?'

$$\mathcal{L}_{\text{task}} = \text{mean}_{\text{decided } r}\ \text{BCEWithLogits}(u_M(r), Y(r)).$$

For A, $u_M$ is the GRU task head. For B/C, $u_M$ is the **logit derived from the joint model's outcome probability**; it must not be an independent unsynchronized head. With budget negatives, this predicts **success under the named protocol**, not unrestricted physical feasibility.

### 10.3 Effect loss: 'did I understand what the intervention changes?'

$$\mathcal{L}_{\text{effect}} = \text{mean}_{\text{fully decided contrasts}}(\widehat{\Delta} - \Delta)^2.$$

The term includes eligible scene effects and repair omissions, and potentially joint interaction residuals if separately defined and all sides decided. No gradient comes from a contrast with an unknown component or mismatched teacher policy.

**A** computes predicted effects by independently rescoring changed initial contexts/rebuilt sequences. **B/C** calculate them from the probabilities of their explicit outcome variables.

### 10.4 Travel-cost loss: 'how far did the actual robot joints move?'

$$\mathcal{L}_{\text{cost}} = \text{mean}_{\text{complete relocation witnesses}}\ \text{Huber}\left(\log(1+\widehat{J}), \log(1+J^*)\right).$$

Huber penalizes prediction errors smoothly while being less sensitive to unusually large errors than plain squared error. The log scale reduces sensitivity to long-tail travel values. It supervises **joint travel**, not endpoint distance. A

negative task may still have a usable J if its relocations succeeded before the target failed. Scene edits must have `cost_mask=false`.

### 10.5 A's sequence-only pool energy loss

Suppose one initial scene has a frozen candidate set $\mathcal{C}$, with both successful and failed proposals. Let $\mathcal{C}_+$ be **all positive repairs** in that decided pool. Define

$$P_A(r \mid \mathcal{C}) = \frac{e^{-E_A^{\text{feas}}(r)/T}}{\sum_{v \in \mathcal{C}} e^{-E_A^{\text{feas}}(v)/T}}, \quad \mathcal{L}_{J,A} = -\frac{1}{|\mathcal{C}_+|} \sum_{r \in \mathcal{C}_+} \log P_A(r \mid \mathcal{C}).$$

The lesson for the model is: 'put probability mass on **all** known successful repairs, not only the first teacher favorite.' Skip all-positive/all-negative pool configurations under the specified guide. This loss uses **feasibility energy without predicted motion cost**, so an otherwise successful long move is not punished simply for being costly.

### 10.6 B/C's true joint Boltzmann loss

For a **fully decided** set of candidate repairs $\mathcal{C}_{\text{full}}$, each repair has an independently observed vector $z^*(r)$ with all its active outcomes known. Let $\mathcal{A}_{\text{full}}$ contain every allowed pair $(v, z)$, including incorrect *hypothetical* binary assignments. Then

$$\mathcal{L}_{J,B/C} = -\frac{1}{|\mathcal{C}_{\text{full},+}|} \sum_{r \in \mathcal{C}_{\text{full},+}} \log \frac{e^{-E_{B/C}(r, z^*(r))/T}}{\sum_{(v,z) \in \mathcal{A}_{\text{full}}} e^{-E_{B/C}(v,z)/T}}.$$

This term does two things at once: it scores candidate repairs **and** the joint consistency of their predicted explanation outcomes. Add individual outcome-field BCE and effect losses so the model also learns the correct outcomes for **negative** candidate repairs rather than focusing only on positive numerators.

**Why the small dataset matters:** P4 currently has one complete recorded B/C outcome pool, and the old slot layout needs correction. That is enough to check exact normalization, nonzero gradients and memorize a toy case. It is **not** sufficient evidence that B/C actually beat A. The proposed small extension must target **multiple complete multi-repair pools**, not simply dozens of individually decided queries.

### 10.7 Confidence, class balance and gradients

Initially, Methodology v1.7 prescribes equal starting supervised loss weights, AdamW learning rate $10^{-3}$, weight decay $10^{-4}$, and width 128. Report 0019 currently gives operational budget negatives label weight **1.0** with a down-weight ablation hook. The PoC should log loss per term, effective labelled sample counts, gradient norms and prevalence by source. Test ablations: **certified-only** versus **certified plus operational**, and perhaps a predeclared lower operational weight. Do not infer confidence weights from test performance.

The most important regression test is this: if all loss masks on an outcome field are false, that field's supervised term must produce **exactly zero gradient**. Another is that the trainable B/C coupling factors must receive nonzero gradients from a complete-pool joint objective in a fixture intentionally constructed to exercise them.

## 11 One worked example from initial scene to gradient

This section uses a **hypothetical teaching example**, not a new measurement; the real pilot references appear separately below.

**Initial scene:** The robot's target is obstructed. Two movable objects A and B are near the workspace. Desired target goal is fixed. Candidate endpoint slots are `A -> a` and `B -> b`.

**Frozen candidate set:**

- $r_0 = []$: no repair;
- $r_1 = [A \to a]$;
- $r_2 = [B \to b]$;
- $r_3 = [A \to a, B \to b]$;
- $r_4 = [B \to b, A \to a]$.

**Hypothetical decided teacher outcomes:** $Y(r_0) = 0, Y(r_1) = 0, Y(r_2) = 0, Y(r_3) = 0, Y(r_4) = 1$. Because $r_3, r_4$ have the same endpoints, the important distinction is **order**. All these results must be independently evaluated; the model cannot assume them from final object positions.

**Hypothetical fixed-robot scene edits:** A only gives 0, B only gives 0, both give 1 (with original 0). Then $\Delta^S_A = 0$, $\Delta^S_B = 0$, $\Delta^S_{A+B} = +1$, and $\kappa = +1$. This is a concrete *definition* of positive interaction, not a universal rule that two blockers always behave this way.

**Hypothetical omissions from** $r_4$: if dropping A leaves a failed repair and dropping B also fails, both omission effects are +1. Every omitted shorter sequence must be replayed from $S_0$.

**What A learns:** endpoint compatibility from the final positions (same for r3/r4), plus GRU evidence that B then A works while A then B does not. Its explanation effects come from repeated task-scoring.

**What B learns:** an order-sensitive prefix pair term for B then A and joint potentials preferring a coherent bundle of original/edited/repair/omission outcomes. It scores all candidate $(r, z)$ states during exact training.

**What C learns:** B's joint intervention consistency plus the GRU's higher-order action-history memory.

**What the optimizer sees:** For every model, the correct label produces BCE gradients on its task factors; for B/C, the observed joint z vector and all competing hypothetical vectors produce additional joint-energy gradients. If measured relocation witnesses exist, cost Huber gradients train $\widehat{J}$. The network never sees the teacher's trajectory or colliding path in its inputs.

**What the sampler does next:** Under each trained model, propose/rank repairs via its sampling energy, under a fixed compute budget. Independently screen/verify the chosen order. Failure in physical verification still overrides the model's low energy.

# 12 Displacement, ordering and robot travel: who decides what?

## 12.1 Is moving objects learned?

**What to move / where / in what order:** yes, these are the **high-level decisions** the neural energy ranks. The inference sampler changes candidate destination assignments, swaps action order, adds a move (birth) or removes one (death). The learned model scores those proposed changes.

**How to execute an individual move in joint space: not learned here**. The external low-level planner chooses grasps and continuous arm trajectories within the fixed requested action order. Its successful trajectory is the evidence witness.

**Whether a hypothetical teleport helps:** measured via separate offline **fixed-robot scene-edit** teacher queries. It is not counted as a robot-executable action.

## 12.2 Is displacement cost learned?

The endpoint displacement of each object is **analytically computed** from known initial/destination positions:

$$C_d(r) = \sum_{(i,k)\in r} \frac{\|p_{ik} - p_{i0}\|_2}{L_W}.$$

There is **no displacement-prediction neural head** in this specification. The model *does* learn a separate **joint-travel predictor** $\widehat{J}_M(r)$ from first-witness motion measurements. These should not be conflated. A short object displacement may require a huge robot-arm detour, and vice versa.

### 12.3 Exactly how is ordering used?

- **A:** GRU updates after each action with ideal prefix-context information.
- **B:** trainable selected-object, adjacent-action and prefix/predecessor interaction potentials, deliberately excluding a GRU.
- **C:** GRU from A *plus* B's explanatory couplings, with B's old order term replaced.

The same-endpoint order test is a clean experiment because endpoint-only static factors cancel; a model must exploit the ordered part of its architecture to distinguish the alternatives.

## 13 The sampler, the optimizer, and final plan selection are different things

**Neural optimizer (training):** AdamW adjusts the trainable model weights to reduce supervised/joint negative log-probability losses. It does not move any real object.

**Boltzmann sampler (inference):** explores a finite space of candidate high-level ordered repairs according to trained sampling energy. It is not a gradient descent optimizer over continuous robot joint trajectories.

**Independent motion verifier:** tries physically executable relocation trajectories for the candidate orders without changing them. It rejects nonexistent witnesses regardless of the learned score.

For A, the sampler uses

$$E_A^{\text{sample}}(r) = E_A^{\text{feas}}(r) + \lambda_d C_d(r) + \lambda_q \widehat{J}_A(r).$$

For B/C, use the **exact z-marginal repair energy** $\bar{E}_{B/C}(r)$ instead of A's feasibility energy and then add the identical analytic displacement and predicted robot travel terms.

At final ranking, after the validation-frozen task-score threshold and exact static screening, ranking is **lexicographic**:

1. **Fewest moved objects**;
2. **Lowest predicted joint travel**;
3. **Least analytic endpoint displacement**;
4. **Lowest learned feasibility/marginal energy**;
5. **Deterministic sequence identity** as the tie break.

Feasibility eligibility comes first. A zero-move but impossible original task must not win automatically merely because it has zero cost.

### 13.1 Sampler moves and mathematical validation

The intended sequence-space kernels are destination Gibbs, action-order swap, birth of a new object/destination/insertion-position and death of an existing action. Birth/death are **asymmetric**, so their Metropolis-Hastings acceptance ratios require the correct forward/reverse proposal probabilities, including object-specific destination counts $K_i$. An unavailable move becomes a self-transition.

For B/C, sample repairs based on **marginalized** $\bar{E}(r)$, then infer $z \mid r$, or use a separately validated joint kernel. Tiny enumerated domains should prove probability row sums, detailed balance and stationary distributions. An animation alone cannot prove correct sampling.

## 14 What happens when the robot actually acts? The one-shot boundary

The model's prediction and the offline data teacher are allowed to differ in *how often they test hypothetical outcomes*.

**Offline data generation** can evaluate numerous alternative repairs and original/edited scenes, each reset to the same root state, to create labels.

**Deployed evaluation** has a stricter sequence:

1. From initial observation alone, score original feasibility and eligible what-if queries; propose repairs through Boltzmann inference.
2. Rank/deduplicate a **frozen** candidate list *before learning target outcomes*.
3. The verifier may try several candidates to find a complete **relocation-only witness**, but may not use an attempted final target failure to choose a different repair.
4. Commit the first suitable complete relocation witness, execute actions in order and check achieved states.
5. Evaluate the final target **at most once** after the committed relocations. Stop on failure; no silent high-level reordering or return to an alternate candidate after seeing the target outcome.

This one-shot boundary prevents the model from looking artificially strong by using teacher feedback from many target attempts at test time.

## 15 The actual A-versus-B-versus-C experiments

| Scientific question | Matched comparison | What must be held fixed |
|---|---|---|
| Does learned repair energy help at all? | A vs geometry-only and uniform ordered repair proposals | Target set, candidate/verifier budgets, eligibility |
| Do explicit intervention outcome variables help beyond GRU? | **A vs C** | GRU, root splits, label access, matched capacity |
| Does recurrent order reasoning help beyond prefix pairs? | **B vs C** | Joint z catalogue, outcome losses, search budget |
| Is B's coupling term useful? | **B vs B-no-coupling** | Data, model size control, optimization budget |
| Is C's coupling term useful? | **C vs C-no-coupling** | GRU, data, training and proposal budgets |
| Does appearance matter given supplied shape/pose? | Each arm vs its geometry-only version | Remove only frozen visual features |
| Do paired intervention labels help? | With versus without effect supervision | Matched training-root and effective label accounting |
| Does order really matter? | Full ordered factor versus trained no-order ablation | Same endpoints, actual opposite-order evidence |
| Do operational negatives change conclusions? | Certified-only vs certified + budget negative | Same roots, transparent masks and label source |

A larger model is not automatically better. C may have more parameters than A; include a capacity-matched A. The goal is to choose the *simplest empirically successful model*, not to preannounce that 'more causal structure' must win.

### 15.1 Primary metrics

**Physical/task success:** proportion of **all eligible initial episodes** whose proposed and independently checked repair truly leads to goal + SAFE_RETREAT; include abstentions, timeouts and aborts in denominator.

**Repair efficiency:** number of moved objects, measured normalized robot travel, endpoint displacement, verifier and target calls, CPU and wall time.

**Prediction quality:** task classification and calibration, static-factor correctness, learned energy ranking over decided candidate pools, cost prediction versus actual witnessed cost.

**Explanations:** signed scene edit and omission effect accuracy, $\kappa$ residual accuracy only where all sides are known, nuisance false-positive rate, proof-grounded versus operational subgroup results.

**Order:** correct ranking of physically decided, same-endpoint order contrasts, plus coverage (how many eligible roots actually yield such contrasts).

**Statistics:** split and bootstrap by independent **root**, never scatter descendants of one scene across train, validation and test. Small development roots cannot justify meaningful population-level confidence intervals.

## 16 What new data do we need before the first neural proof of concept?

We should **not** wait for 40 roots. We already have enough to make the code run and test gradients. But a little more independently decided real data will make the PoC substantially more informative, especially for B/C.

The proposed immediate extension is **eight newly generated independent 2D roots**, collected in two authorized subbatches of four, with a **predeclared maximum 12 unique teacher queries per root** (at most 96 queries; not 96 guaranteed decided labels). The up-to-12-query catalogue explicitly includes one **clean-base baseline witness query** plus one **obstructed-scene original task**; the remaining queries are a frozen subset of two singleton repairs, two same-endpoint two-action orders, three fixed-robot singleton/joint edits, one nuisance edit, and up to two destination alternatives. Genuine distinct omissions reuse the singleton queries where equivalent; otherwise consume one of the 12 slots, reducing alternatives. The baseline and obstructed scenes must **never share one joint-outcome vector**. A no-op omission is not a unique query. **Record every attempted root, query and undecided outcome.**

**Targeted balance** (design intentions, not guaranteed observed successes): small 2D J0 original-feasible controls, r19-like access/order variants, dual-obstruction effects, alternative destinations, nuisance/harmful edits, and cost-divergent repairs. Do **not** choose examples by looking at future outcomes; derive proposal catalogues before teacher calls.

**Frozen splits before descendants:** four fresh development roots, two tiny validation roots and two diagnostic holdout roots. Historical P4 and G4C development roots can be used for **training/debugging only**, never relabelled as held out. With two holdout roots the only defensible claim is *mechanical sanity on untouched examples*, not statistical generalization.

**Desired evidence gates before interpreting learning behavior:** at least four distinct complete B/C pools across at least three independent roots (including P4 where applicable); at least two contrasting two-action orders; at least two nonzero fully decided scene/omission interaction calculations with clear scope; at least six first-witness relocation travel labels and at least two nonempty complete witnesses per independent scene category represented. These are **stretch/interpretability targets**, not grounds for fabricating or discarding unknowns. If they fail, run the PoC code and explicitly mark the affected scientific hypothesis *not yet testable*.

**Shared teacher policy:** keep `2D-ABC-OPNEG-v1` unchanged (three pinned seeds), rather than introducing a new seed policy mid-extension. Its operational negatives are protocol-relative; report source proportions. Preserve the exact full 3R, one-shot and no-leakage conditions.

**Proposed capped compute envelope, subject to explicit authorization:** max eight new independent roots; max 96 unique teacher queries, max 288 seed attempts; aggregate worker+coordinator CPU **4,800 seconds** and wall **7,200**

**seconds** with one worker; original P4 per-seed CPU 250 seconds and per-query max 900 seconds, with process-tree guard; static admission up to **300 CPU-seconds**, counted within or explicitly separately as predeclared. Stop before launch unless the implementation can enforce every aggregate guard, including subprocesses. These numbers are proposed controls, **not estimates of actual duration**; P4's 6.5 CPU-s per admitted label came from deliberately selected easy development roots and cannot be extrapolated as a guarantee.

**Stop/adapt after the first four roots:** stop the extension if a fatal witness/certificate contradiction, source/hash mismatch, or process-tree breach occurs; if more than 25% of planned queries are unknown after four roots, halt new teacher collection and diagnose. If the first four roots produce no complete joint pool or no distinct order alternatives, still proceed to tiny-data model smoke testing rather than blindly spending the remaining budget on near-duplicate scenes. This is a **proposal**, not a completed gate.

**Prerequisite before any B/C smoke:** correct the duplicated no-op omission in a new versioned z-layout, retain old evidence, verify masks and partition invariance, and rerun the small G4B sampler tests. A full code change and new model training are not yet reported.

# 17 Exactly what would the proof-of-concept phase do?

A PoC is **not** a publication-level comparison. It is an implementation and learnability check using small real data and synthetic mathematical fixtures.

**Phase 0 — data and math audit.** Correct the z-layout; verify all recorded outcomes in the candidate pool use the same teacher policy; rebuild loss masks and cost labels; ensure anchor appearance stays frozen for hypothetical scene edits. Verify independent-root splits.

**Phase 1 — factor overfit.** Train shared object projections and static unary/pair heads on decided static examples. Check losses fall, output probability calibration behaves, pair invariance holds, and validation code never sees hidden teacher fields.

**Phase 2 — A actual training.** Train the full A sequence GRU, task, cost and effect heads on all permitted development examples. Verify training loss falls, `g4c_r19` witnessed order is favored over its certified-inaccessible swap, and J prediction follows first-witness labels. This is in-sample memorization, not generalization.

**Phase 3 — B actual joint training.** Construct an exact fully enumerated z microdomain (seven active fields for the canonical two-move layout), verify the partition and gradients numerically, train B's full potentials jointly and test that the correct observed outcome vectors receive higher probability. Compare with B-no-coupling and ensure the coupling term genuinely changes when trained.

**Phase 4 — C actual joint + GRU training.** Repeat the exact joint checks for C and train its GRU order factor with the same explanation data. Inspect that GRU and coupling factors both receive gradients. Compare learned scores on the identical available decided repair pairs.

**Phase 5 — sampler and ranking.** Feed all three learned energies into the correctly verified finite Gibbs/swap/birth/death sampler. Record acceptance rates, uniqueness, finite-pool rankings, and exact static/verifier outcomes **without changing the one-shot target boundary**. Use cached teacher outcomes for *offline label comparison* only; never let them choose the ranked proposal during evaluation.

**Phase 6 — minimal ablations.** Run A without GRU/order; B without coupling; C without coupling; optionally A with matched capacity. Run a budget-negative sensitivity check using certified-only versus combined records. These are sanity ablations at this size, not definitive statistical findings.

### 17.0.1 Strict success levels

**Engineering PASS:** three genuine trainable architectures with nonzero intended gradients, correct masks and joint probability normalization, unit-tested sampler, saved checkpoints and reproducible real tiny-data training.

**Behavioral PoC PASS:** on development examples, A/C distinguish decided order swaps; B/C fit at least one complete joint pool with actual joint loss; no target/teacher leakage; learned proposal ranking can be audited against stored independent witnesses/certificates. Distinguish in-sample memorization from held-out performance.

**Scientific hypothesis TESTABLE:** multiple independent, diverse, fully decided joint pools and order contrasts exist. The prepared mini-extension might suffice for *initial* validation; if not, identify exactly which contrast is missing and collect **that**, not another indiscriminate 40 roots.

**Scientific superiority established: not from this PoC**. Requires matched held-out roots, repeated seeds, strong analytic baselines, outcome and compute accounting, and the later G5/G6 comparison.

## 18 Common misconceptions and how to answer them

**'Are we learning collisions?'** Not directly as an oracle. Local compatibility and complete-plan outcome patterns are learned; exact collision checking remains external. Feeding collision checker outputs as learner inputs would leak the answer.

**'Are we learning how the arm should move?'** No low-level trajectories or controller. We rank ordered high-level relocations; robot motion is planned and independently verified.

**'Does energy minimize the number of objects moved?'** Move count is an explicit hard budget and later **lexicographic priority**, not a learned differentiable objective. The Boltzmann sampling energy uses task energy plus displacement and predicted joint travel.

**'Is the object displacement learned?'** No, it is calculated exactly. What we learn is **which destination and sequence is likely to work** and a model of the resulting robot's joint travel.

**'Are interventions used to choose moves or to learn causes?'** They are predeclared what-if **data queries** whose independently measured outcomes supervise effect prediction; B/C explicitly model their outcomes jointly with candidate repairs. At inference the model predicts those outcomes from initial context, not by secretly executing the intervention.

**'Does B learn a causal graph?'** No. It learns factor potentials over explicitly defined what-if task outcomes. A large potential is not a discoverable causal arrow or a proof of unique physical necessity.

**'Why B if A can already estimate effects?'** A estimates effects by subtracting two scores trained mostly separately. B tries to learn a coherent **joint distribution** that couples those questions with the proposed repair. Whether this is an advantage is the hypothesis.

**'Why C if B already understands order?'** B has prefix/pair ordering factors. C uses a higher-order GRU memory while retaining B's explicit joint outcome coupling. Whether this helps on three-step dependencies is the hypothesis.

**'What if the planner times out?'** Under `2D-ABC-0PNEG-v1`, three completed clean seed runs can create a **budget negative** for a new teacher record. A guard-killed or incomplete run is still UNKNOWN. Operational negative does **not** mean mathematical impossibility.

**'Can we release a dataset with zero UNKNOWN rows?'** We may publish a binary-supervised subset containing only decided rows, with all attempted and unresolved queries retained in the audit and coverage denominator. We cannot guarantee the teacher will decide every randomly constructed scene; filtering can introduce bias and must be reported.

**'Is lower energy proof of a causal relationship?'** No. Independent scene-effect and omission measurements ground specific scoped comparisons; learned energy makes predictions about them.

# 19 Decision and next implementation order

**Recommendation:** carry out a **small, diversity-targeted 8-root D3-lite extension**, not the proposed 40-root collection; but **first** correct the B/C duplicate-outcome layout and re-run G4B. Run the extension in a **4-root checkpoint + optional 4-root continuation**, then implement and train **A/B/C proof-of-concept models immediately**. Use the already collected P4 cases and earlier compatible development data for wiring and overfitting. If the extension fails to create multiple fully decided B/C joint pools, train the models anyway and explicitly label the coupling-generalization question as untested.

Don't start 3D, hardware or full G5/G6 comparison on the strength of a tiny overfit test. Don't change historical labels, use old G4C pilot roots as hidden test roots, declare operational failures mathematically impossible, or pretend an offline teleport is an executable robot repair.

**The real research decision later is whether the added explicit joint causal-outcome representation of B/C leads to better independently verified repair choices and effect predictions than A and strong geometry-only controls, under matched data, computation and parameter-capacity analysis. A null result is valid.**

# 20 Appendix A. Exact source basis and status

**Authoritative scientific architecture:** `latent_causal_tamp_methodology_v1_7.md`, especially *Scientific decision and hypotheses*, *Retained physical problem*, *Shared data/interventions*, *Architecture A/B/C*, *Training objectives*, *Search* and *Evaluation*.

**Authoritative engineering boundaries:** `latent_causal_tamp_implementation_guide_v1_9.md`, especially *One shared schema*, *Sampler*, *Training*, *One-shot inference*, *Fixture and dataset plan* and *Stopping criteria*.

**Implemented post-spec policy/evidence:** `0019_G4C_LF_P4_operational_negatives_pilot.md`, pushed as `0bd8fdd` as reported in the conversation; particularly the policy addendum, P4 outcome labels, zero unknown calibration, regression closure and duplicated B/C omission slot.

**Supporting provenance:** `0018_G4C_D2_P3a_sector_certificate_results.md` (sound fixed-start pick certificate, r19 access contrast, regression first incomplete then closed in P4); `0017_G4C_D2_P2_certificate_soundness_review.md` (reason for the scope); `0016_G4C_D2_P1_poc_s.md` (limitations of earlier proof-first data).

**What's new in this report rather than already established:** the proposed **8-root/4+4 extension and budget**, small-root split, checkpoint gates, alias audit suggestion, and the explanation examples. These are recommendations only, not changes to the frozen repository or claims of measured performance.